Goutham Solasa

Advanced Big Data and Data Mining

Lab 1: Data Visualization, Data Preprocessing, and Statistical Analysis Using Python in Jupyter Notebook

Step 1: Data Collection

Importing Libraries and Loading the Dataset

The necessary libraries were imported to facilitate data handling, calculations and visualisation. Data was successfully loaded and the first 5 records were examined.

In [1]:
```
# Importing required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

# Suppressing warning messages
warnings.filterwarnings('ignore')

# Loading the dataset
df = pd.read_csv("E:\\SuperMarket Analysis.csv")

# Displaying the first five rows
print("First Five Rows of Dataset")
display(df.head())
```

```
First Five Rows of Dataset
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.

Displaying Dataset Information

The dataset information was presented for an analysis of the missingness of data, data types, and columns. The overall dataset structure was checked before further data analysis.

In [2]:
```
# Displaying dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Invoice ID               1000 non-null   object 
 1   Branch                   1000 non-null   object 
 2   City                     1000 non-null   object 
 3   Customer type            1000 non-null   object 
 4   Gender                   1000 non-null   object 
 5   Product line             1000 non-null   object 
 6   Unit price               1000 non-null   float64
 7   Quantity                 1000 non-null   int64  
 8   Tax 5%                   1000 non-null   float64
 9   Sales                    1000 non-null   float64
 10  Date                     1000 non-null   object 
 11  Time                     1000 non-null   object 
 12  Payment                  1000 non-null   object 
 13  cogs                     1000 non-null   float64
 14  gross margin percentage  1000 non-null   float64
 15  gross income             1000 non-null   float64
 16  Rating                   1000 non-null   float64
dtypes: float64(7), int64(1), object(9)
memory usage: 132.9+ KB
```

Step 2: Data Visualization

Converting Date Column Format

The Date column was changed to a date and time format for more convenient analysis. The data became suitable for performing time-based operations.

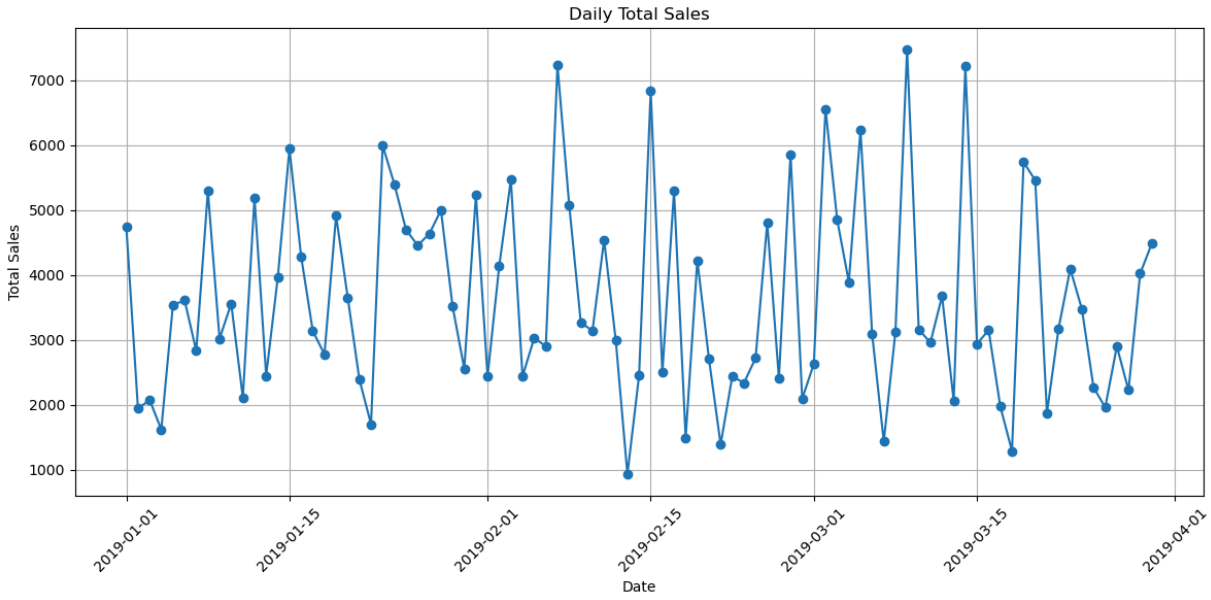
In [3]:
```
# Converting date column format
df['Date'] = pd.to_datetime(df['Date'])
```

Creating a Line Plot for Daily Total Sales

Sales values were grouped and the total sales for each date were determined. A line graph was plotted to show the trend of sales over time.

In [4]:
```
# Calculating daily total sales
daily_sales = df.groupby('Date')['Sales'].sum()

# Creating figure size
plt.figure(figsize=(12,6))

# Drawing line plot
plt.plot(daily_sales.index, daily_sales.values, marker='o')

# Adding plot title
plt.title('Daily Total Sales')

# Adding horizontal label
```

```
plt.xlabel('Date')

# Adding vertical label
plt.ylabel('Total Sales')

# Rotating horizontal labels
plt.xticks(rotation=45)

# Displaying grid lines
plt.grid(True)

# Adjusting plot layout
plt.tight_layout()

# Displaying the plot
plt.show()
```



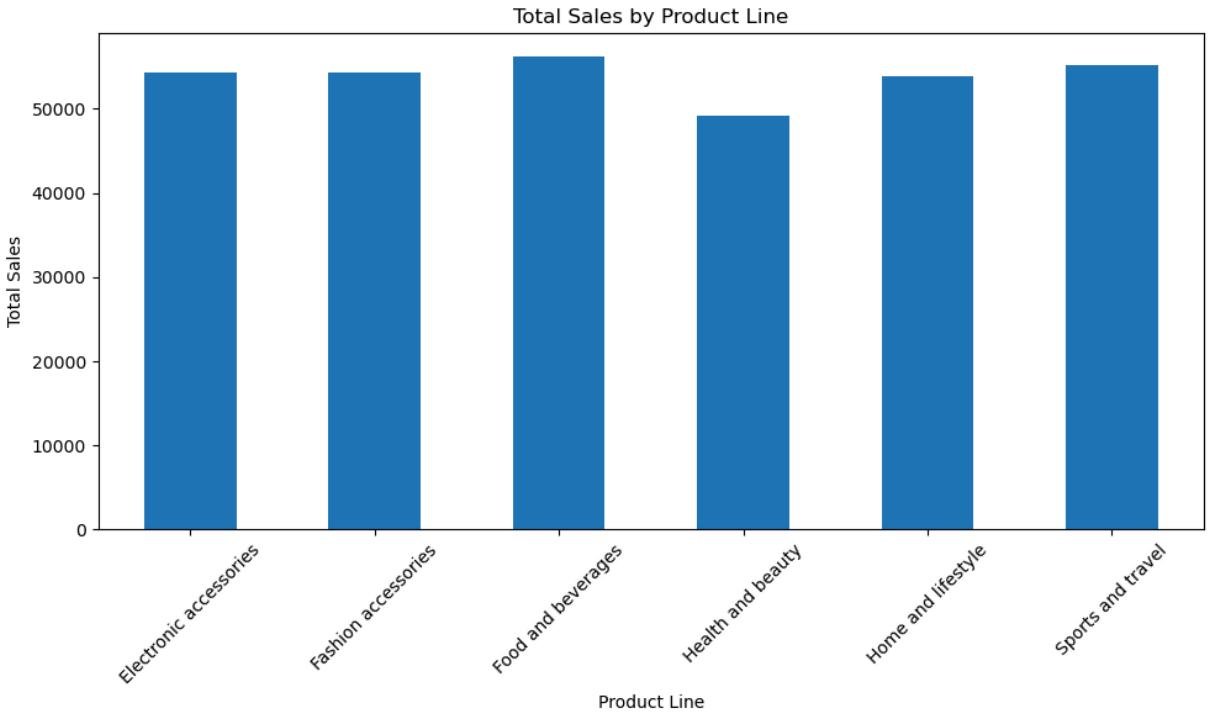
The total sales per day had some noticeable fluctuations over the period of observation, which meant that the sales performance pattern wasn't consistent from one day to the next. There were several sharp peaks that signified days with extremely high sales, and a few deep dips that signified days with lower sales activity. The sale values for most days were in a moderate range with a few extreme rises and falls. There was no discernible trend in the daily sales, indicating that the sales data for those days was not consistently increasing or decreasing.

Creating a Bar Chart for Sales by Product Line

The sales values were added to get the total sales for each product line. A total bar chart was produced to make comparisons between the total sales of products.

In [5]:
```
# Calculating sales by product line
product_sales = df.groupby('Product line')['Sales'].sum()

# Creating figure size
```

```
plt.figure(figsize=(10,6))

# Drawing bar chart
product_sales.plot(kind='bar')

# Adding chart title
plt.title('Total Sales by Product Line')

# Adding horizontal label
plt.xlabel('Product Line')

# Adding vertical label
plt.ylabel('Total Sales')

# Rotating horizontal labels
plt.xticks(rotation=45)

# Adjusting chart layout
plt.tight_layout()

# Displaying the chart
plt.show()
```

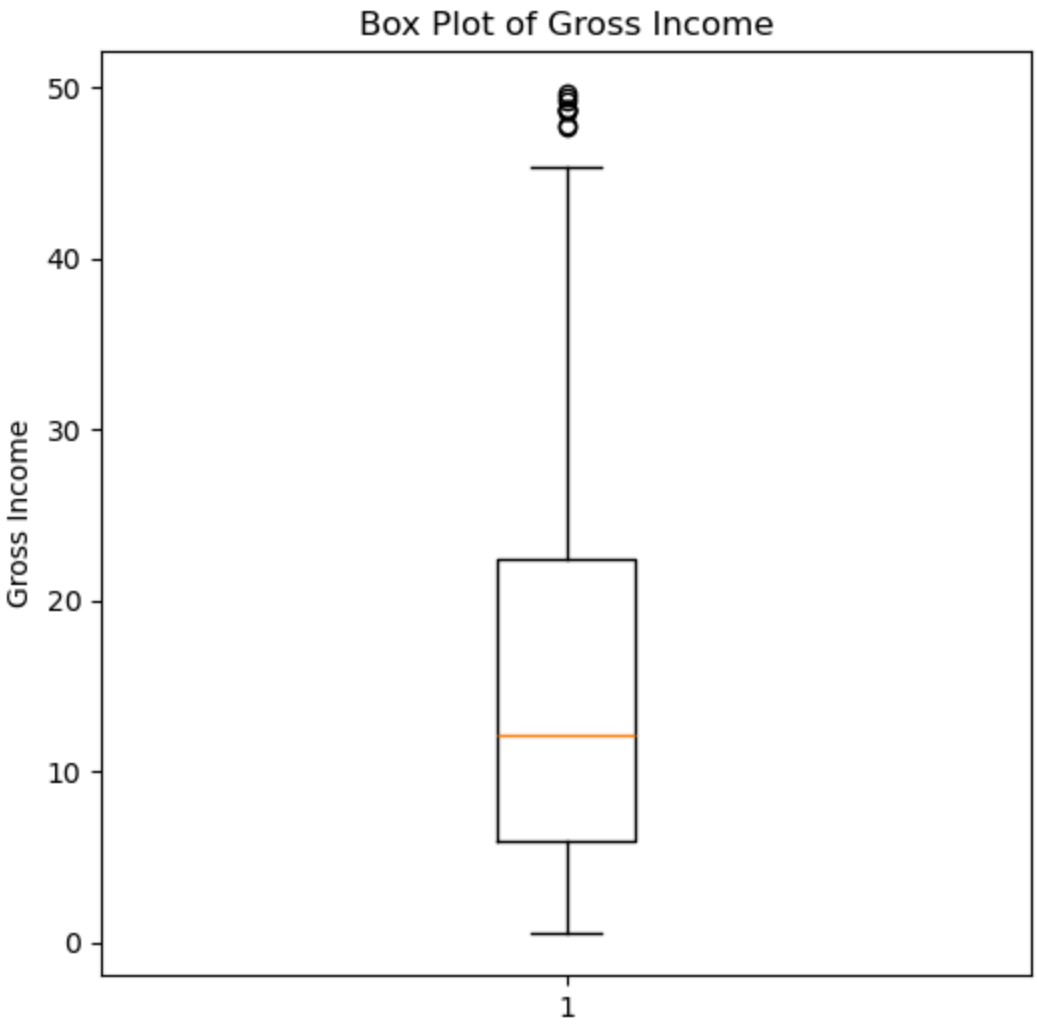


Total sales were relatively consistent by product line, suggesting an even distribution of sales by product line. The highest total sales were in the Food and beverages category and the lowest sales were in the Health and beauty category. The sales values of electronic accessories, fashion accessories, home and lifestyle and sports and travel were similar with minimal differences. The graph showed that, overall, most product lines contributed about equally with none having a big difference in performance.

Creating a Box Plot for Gross Income

A box plot was used to show the gross income values. The chart identified the distribution of values and/or identified any unusual values.

In [6]:
```
# Creating figure size
plt.figure(figsize=(6,6))

# Drawing box plot
plt.boxplot(df['gross income'])

# Adding plot title
plt.title('Box Plot of Gross Income')

# Adding vertical label
plt.ylabel('Gross Income')

# Displaying the plot
plt.show()
```



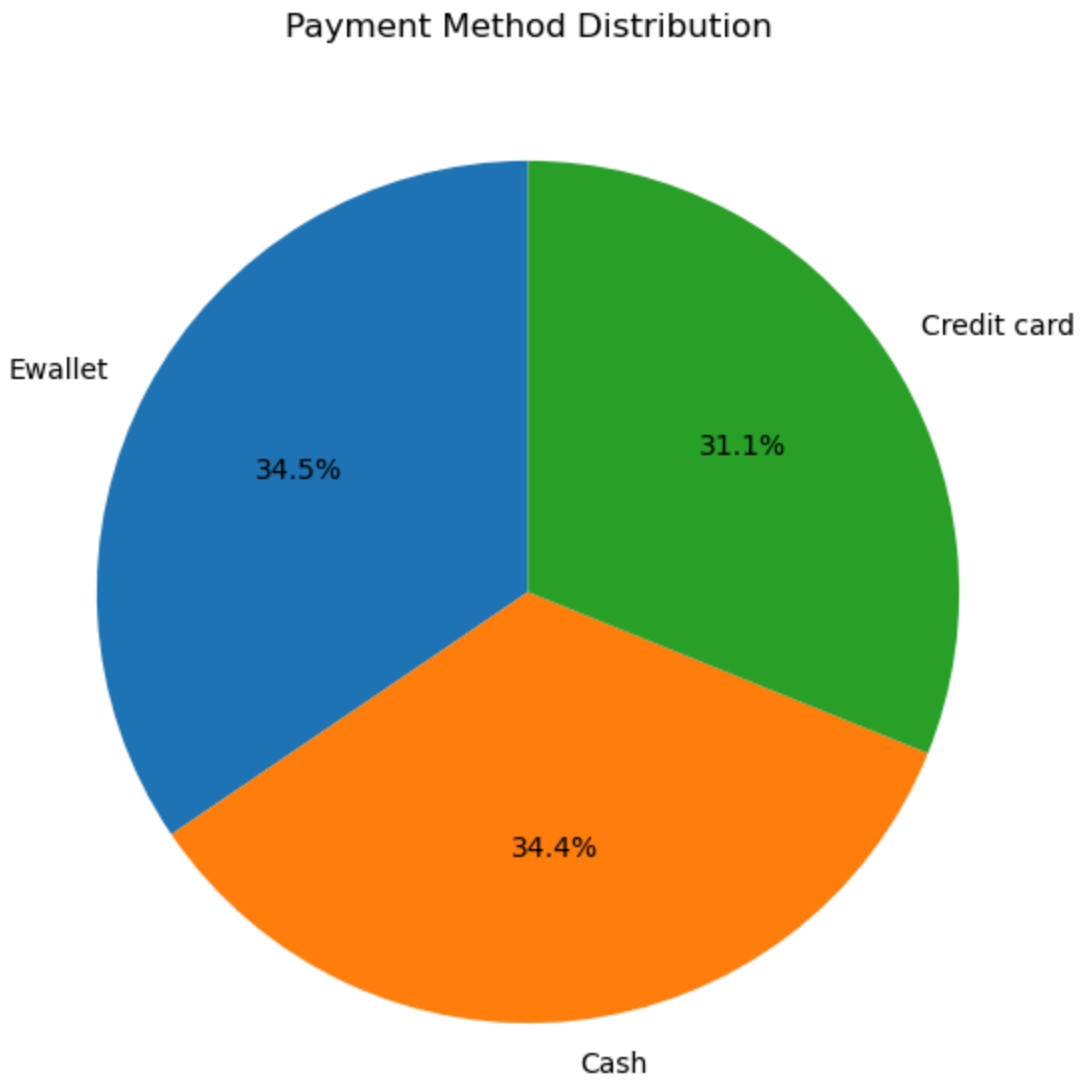
The median gross income was near the center of the distribution and the middle 50% was moderately spread. The top whisker was much longer than the bottom whisker, suggesting that there was a lot of variation in the values of the gross income on the top. There were a few values above the upper whisker that may be considered outliers if they represent people

with very high gross income. Overall, the distribution was positively skewed, as there were more high values than low values.

Creating a Pie Chart for Payment Method Distribution

Data was used to calculate the number of transactions for each payment method. The information was presented in a pie chart, in which each section is a percentage of the whole.

In [7]:
```
# Calculating payment method counts
payment_counts = df['Payment'].value_counts()

# Creating figure size
plt.figure(figsize=(7,7))

# Drawing pie chart
plt.pie(payment_counts,
        labels=payment_counts.index,
        autopct='%1.1f%%',
        startangle=90)

# Adding chart title
plt.title('Payment Method Distribution')

# Displaying the chart
plt.show()
```



The distribution of payment methods was relatively even, with no significant difference between the three categories, suggesting a fairly uniform customer preference. Ewallet had the largest percentage at around 34.5 percent while cash came in close behind at around 34.4 percent. The smallest share of transactions were via credit card, around 31.1 percent, but this was barely 1 percent of the total. Overall, the chart shows that all of the payment methods are widely used and no single payment method dominates.

Step 3: Data Preprocessing

1. Handling Missing Values

Detecting Missing Values

The missing values for each column were determined and the number of missing values counted. The results were presented so that it could be identified which columns needed data cleaning.

In [8]:
```
# Displaying missing values in each column
print("Missing Values in Each Column")
print(df.isnull().sum())
```

```
Missing Values in Each Column
Invoice ID                 0
Branch                     0
City                       0
Customer type              0
Gender                     0
Product line               0
Unit price                 0
Quantity                   0
Tax 5%                     0
Sales                      0
Date                       0
Time                       0
Payment                    0
cogs                       0
gross margin percentage    0
gross income               0
Rating                     0
dtype: int64
```

All the features have a count of 0 for missing values, meaning there are no missing values in the dataset. This means that there are no missing values that need to be treated or imputed to the set of data in order for them to be analyzed.

2. Outlier Detection and Removal (IQR)

Calculating Outlier Limits Using IQR

Sales values were used to find the first quartile, third quartile and interquartile range. The lower and upper limits were then calculated to see if there were any outliers present.

In [9]:
```
# Calculating first quartile
Q1 = df['Sales'].quantile(0.25)

# Calculating third quartile
Q3 = df['Sales'].quantile(0.75)

# Calculating interquartile range
IQR = Q3 - Q1

# Displaying first quartile
print("First Quartile (Q1):", Q1)

# Displaying third quartile
print("Third Quartile (Q3):", Q3)

# Displaying interquartile range
print("Interquartile Range (IQR):", IQR)
```

```
# Calculating lower limit
lower_limit = Q1 - 1.5 * IQR

# Calculating upper limit
upper_limit = Q3 + 1.5 * IQR

# Displaying lower limit
print("Lower Limit:", lower_limit)

# Displaying upper limit
print("Upper Limit:", upper_limit)
```

```
First Quartile (Q1): 124.422375
Third Quartile (Q3): 471.35024999999996
Interquartile Range (IQR): 346.927875
Lower Limit: -395.9694375
Upper Limit: 991.7420625
```

Identifying and Removing Outliers

The outliers were determined by comparing the sale values to the calculated limits. The results of the removal of the outliers were shown and the size of the data set before and after the removal of the outliers was displayed.

In [10]:
```
# Identifying outliers
outliers = df[(df['Sales'] < lower_limit) | (df['Sales'] > upper_limit)]

# Displaying identified outliers
print("\nIdentified Outliers")
display(outliers)

# Removing outliers
no_outliers = df[(df['Sales'] >= lower_limit) &
                 (df['Sales'] <= upper_limit)]

# Displaying dataset shape before removing outliers
print("Dataset Shape Before Removing Outliers:", df.shape)

# Displaying dataset shape after removing outliers
print("Dataset Shape After Removing Outliers:", no_outliers.shape)
```

```
Identified Outliers
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
166	234-65-2137	Giza	Naypyitaw	Normal	Male	Home and lifestyle	95.58	10	47.790	10
167	687-47-8271	Alex	Yangon	Normal	Male	Fashion accessories	98.98	10	49.490	10
350	860-79-0874	Giza	Naypyitaw	Member	Female	Fashion accessories	99.30	10	49.650	10
357	554-42-2417	Giza	Naypyitaw	Normal	Female	Sports and travel	95.44	10	47.720	10
422	271-88-8734	Giza	Naypyitaw	Member	Female	Fashion accessories	97.21	10	48.605	10
557	283-26-5248	Giza	Naypyitaw	Member	Female	Food and beverages	98.52	10	49.260	10
699	751-41-9720	Giza	Naypyitaw	Normal	Male	Home and lifestyle	97.50	10	48.750	10
792	744-16-7898	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.37	10	48.685	10
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.690	10

```
Dataset Shape Before Removing Outliers: (1000, 17)
Dataset Shape After Removing Outliers: (991, 17)
```

3. Data Reduction

Performing Data Reduction

Before reducing, the size of the original data set was shown. A random sample 30% of the records was done and the reduced data set is shown.

In [11]:
```
# Displaying original dataset shape
print("Original Dataset Shape")
print(no_outliers.shape)

# Selecting random sample
```

```
no_outliers_sample = no_outliers.sample(frac=0.30, random_state=42)

# Displaying sampled dataset shape
print("Sampled Dataset Shape")
print(no_outliers_sample.shape)

# Displaying first five sampled records
display(no_outliers_sample.head())
```

```
Original Dataset Shape
(991, 17)
Sampled Dataset Shape
(297, 17)
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
215	802-43-8934	Alex	Yangon	Normal	Male	Home and lifestyle	18.28	1	0.9140	1
333	442-48-3607	Alex	Yangon	Member	Male	Food and beverages	23.48	2	2.3480	4
506	387-49-4215	Cairo	Mandalay	Member	Female	Sports and travel	48.50	3	7.2750	15
311	181-94-6432	Giza	Naypyitaw	Member	Male	Fashion accessories	69.33	2	6.9330	14
88	504-35-8843	Alex	Yangon	Member	Female	Sports and travel	42.47	1	2.1235	4

Reducing Dataset Columns

Some less relevant columns of the sampled data were dropped to make the number of features smaller. The updated data size and the first few records were then shown.

In [12]:
```
# Removing less relevant columns
df_reduced = no_outliers_sample.drop(columns=[
    'Invoice ID',
    'gross margin percentage',
    'Tax 5%'
])

# Displaying reduced dataset shape
print("Dataset After Dropping Columns")
print(df_reduced.shape)
```

```
# Displaying first five records
display(df_reduced.head())
```

```
Dataset After Dropping Columns
(297, 14)
```

	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Sales	Date	
215	Alex	Yangon	Normal	Male	Home and lifestyle	18.28	1	19.1940	2019-03-22	3
333	Alex	Yangon	Member	Male	Food and beverages	23.48	2	49.3080	2019-03-14	11
506	Cairo	Mandalay	Member	Female	Sports and travel	48.50	3	152.7750	2019-01-08	12
311	Giza	Naypyitaw	Member	Male	Fashion accessories	69.33	2	145.5930	2019-02-05	7
88	Alex	Yangon	Member	Female	Sports and travel	42.47	1	44.5935	2019-01-02	4

4. Data Scaling and Discretization

Applying Min-Max Scaling

The selected numerical columns were prepared for scaling by creating a copy of the dataset. The values were placed in a common range by using Min-Max scaling and then shown in the original and scaled ranges.

In [13]:
```
# Importing scaling library
from sklearn.preprocessing import MinMaxScaler

# Creating dataset copy
df_scaled = df_reduced.copy()

# Selecting columns for scaling
scale_columns = ['Unit price',
                 'Quantity',
                 'Sales',
                 'gross income',
                 'Rating']

# Displaying values before scaling
print("Before Scaling")
display(df_scaled[scale_columns].head())

# Creating scaler
scaler = MinMaxScaler()

# Applying Min Max scaling
df_scaled[scale_columns] = scaler.fit_transform(df_scaled[scale_columns])
```

```
# Displaying values after scaling
print("After Min Max Scaling")
display(df_scaled[scale_columns].head())
```

```
Before Scaling
```

	Unit price	Quantity	Sales	gross income	Rating
215	18.28	1	19.1940	0.9140	8.3
333	23.48	2	49.3080	2.3480	7.9
506	48.50	3	152.7750	7.2750	6.7
311	69.33	2	145.5930	6.9330	9.7
88	42.47	1	44.5935	2.1235	5.7

```
After Min Max Scaling
```

	Unit price	Quantity	Sales	gross income	Rating
215	0.086786	0.000000	0.006421	0.006421	0.716667
333	0.145017	0.111111	0.038503	0.038503	0.650000
506	0.425196	0.222222	0.148731	0.148731	0.450000
311	0.658455	0.111111	0.141080	0.141080	0.950000
88	0.357671	0.000000	0.033480	0.033480	0.283333

Performing Rating Discretization

The rating values were classified into low, medium, and high ranges. The rating values and the categories were then presented for review.

In [14]:
```
# Creating rating categories
df_scaled['Rating Category'] = pd.cut(
    df['Rating'],
    bins=[0,5,7,10],
    labels=['Low','Medium','High']
)

# Displaying dataset after discretization
print("Dataset After Discretization")
display(df_scaled[['Rating','Rating Category']].head(15))
```

```
Dataset After Discretization
```

	Rating	Rating Category
215	0.716667	High
333	0.650000	High
506	0.450000	Medium
311	0.950000	High
88	0.283333	Medium
540	0.850000	High
282	0.416667	Medium
107	0.566667	High
59	0.883333	High
511	0.350000	Medium
363	0.533333	High
450	0.450000	Medium
70	0.933333	High
96	0.183333	Medium
866	0.416667	Medium

Step 4: Statistical Analysis

1. General Overview of Data

Displaying General Dataset Information

The overall dataset information was presented to investigate the structure of the data and the data types. The summary also proved useful for determining the available columns and missing values.

In [15]:
```
# Displaying dataset information
print("Dataset Information")
df.info()
```

```
Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Invoice ID               1000 non-null   object
 1   Branch                   1000 non-null   object
 2   City                     1000 non-null   object
 3   Customer type            1000 non-null   object
 4   Gender                   1000 non-null   object
 5   Product line             1000 non-null   object
 6   Unit price               1000 non-null   float64
 7   Quantity                 1000 non-null   int64
 8   Tax 5%                   1000 non-null   float64
 9   Sales                    1000 non-null   float64
 10  Date                     1000 non-null   datetime64[ns]
 11  Time                     1000 non-null   object
 12  Payment                  1000 non-null   object
 13  cogs                     1000 non-null   float64
 14  gross margin percentage  1000 non-null   float64
 15  gross income             1000 non-null   float64
 16  Rating                   1000 non-null   float64
dtypes: datetime64[ns](1), float64(7), int64(1), object(8)
memory usage: 132.9+ KB
```

Displaying Statistical Summary

The statistical summary of the numerical columns was presented to give an overview of the distribution of the data. Items summarized were: count, minimum, maximum, average and spread of values.

In [16]:
```
# Displaying statistical summary
print("\nStatistical Summary")
display(df.describe())
```

```
Statistical Summary
```

	Unit price	Quantity	Tax 5%	Sales	Date	cogs	gros pe
count	1000.000000	1000.000000	1000.000000	1000.000000	1000	1000.00000	1.000
mean	55.672130	5.510000	15.379369	322.966749	2019-02-14 00:05:45.600000	307.58738	4.761
min	10.080000	1.000000	0.508500	10.678500	2019-01-01 00:00:00	10.17000	4.761
25%	32.875000	3.000000	5.924875	124.422375	2019-01-24 00:00:00	118.49750	4.761
50%	55.230000	5.000000	12.088000	253.848000	2019-02-13 00:00:00	241.76000	4.761
75%	77.935000	8.000000	22.445250	471.350250	2019-03-08 00:00:00	448.90500	4.761
max	99.960000	10.000000	49.650000	1042.650000	2019-03-30 00:00:00	993.00000	4.761
std	26.494628	2.923431	11.708825	245.885335	NaN	234.17651	6.13

2. Central Tendency Measures

Calculating Central Tendency Measures

The numerical columns were chosen for computing the primary measures of central tendency. The minimum value, maximum value, mean, median and mode were determined and presented in a summary table.

In [17]:
```
# Selecting numerical columns
numeric_columns = df.select_dtypes(include=['int64', 'float64'])

# Calculating central tendency measures
central_tendency = pd.DataFrame({
    'Minimum': numeric_columns.min(),
    'Maximum': numeric_columns.max(),
    'Mean': numeric_columns.mean(),
    'Median': numeric_columns.median(),
    'Mode': numeric_columns.mode().iloc[0]
})

# Displaying central tendency measures
print("Central Tendency Measures")
display(central_tendency)
```

```
Central Tendency Measures
```

	Minimum	Maximum	Mean	Median	Mode
Unit price	10.080000	99.960000	55.672130	55.230000	83.770000
Quantity	1.000000	10.000000	5.510000	5.000000	10.000000
Tax 5%	0.508500	49.650000	15.379369	12.088000	4.154000
Sales	10.678500	1042.650000	322.966749	253.848000	87.234000
cogs	10.170000	993.000000	307.587380	241.760000	83.080000
gross margin percentage	4.761905	4.761905	4.761905	4.761905	4.761905
gross income	0.508500	49.650000	15.379369	12.088000	4.154000
Rating	4.000000	10.000000	6.972700	7.000000	6.000000

3. Dispersion Measures

Calculating Dispersion Measures

The columns containing numbers were analyzed to determine the dispersion of the values. The ranges, quartiles and interquartiles, variance and standard deviation were obtained and presented.

In [18]:
```
# Calculating dispersion measures
dispersion = pd.DataFrame({
    'Range': numeric_columns.max() - numeric_columns.min(),
    'Q1': numeric_columns.quantile(0.25),
    'Q3': numeric_columns.quantile(0.75),
    'IQR': numeric_columns.quantile(0.75) - numeric_columns.quantile(0.25),
    'Variance': numeric_columns.var(),
    'Standard Deviation': numeric_columns.std()
})

# Displaying dispersion measures
print("Dispersion Measures")
display(dispersion)
```

```
Dispersion Measures
```

	Range	Q1	Q3	IQR	Variance	Standard Deviation
Unit price	89.8800	32.875000	77.935000	45.060000	7.019653e+02	2.649463e+01
Quantity	9.0000	3.000000	8.000000	5.000000	8.546446e+00	2.923431e+00
Tax 5%	49.1415	5.924875	22.445250	16.520375	1.370966e+02	1.170883e+01
Sales	1031.9715	124.422375	471.350250	346.927875	6.045960e+04	2.458853e+02
cogs	982.8300	118.497500	448.905000	330.407500	5.483864e+04	2.341765e+02
gross margin percentage	0.0000	4.761905	4.761905	0.000000	3.759526e-27	6.131498e-14
gross income	49.1415	5.924875	22.445250	16.520375	1.370966e+02	1.170883e+01
Rating	6.0000	5.500000	8.500000	3.000000	2.953518e+00	1.718580e+00

4. Correlation Analysis

Performing Correlation Analysis

The correlation values were used to measure the relationships among the numerical columns. The correlation matrix was calculated and displayed for comparison of the variables.

In [19]:
```
# Calculating correlation matrix
correlation_matrix = numeric_columns.corr()

# Displaying correlation matrix
print("Correlation Matrix")
display(correlation_matrix)
```

```
Correlation Matrix
```

	Unit price	Quantity	Tax 5%	Sales	cogs	gross margin percentage	gross income	I
Unit price	1.000000	0.010778	0.633962	0.633962	0.633962	NaN	0.633962	-0.0
Quantity	0.010778	1.000000	0.705510	0.705510	0.705510	NaN	0.705510	-0.0
Tax 5%	0.633962	0.705510	1.000000	1.000000	1.000000	NaN	1.000000	-0.0
Sales	0.633962	0.705510	1.000000	1.000000	1.000000	NaN	1.000000	-0.0
cogs	0.633962	0.705510	1.000000	1.000000	1.000000	NaN	1.000000	-0.0
gross margin percentage	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
gross income	0.633962	0.705510	1.000000	1.000000	1.000000	NaN	1.000000	-0.0
Rating	-0.008778	-0.015815	-0.036442	-0.036442	-0.036442	NaN	-0.036442	1.0

In [ ]:

In [ ]: